



Building Decoupled Architectures

AWS Academy Cloud
Architecting



Introduction

Building Decoupled Architectures

Module objectives

This module prepares you to do the following:

- Differentiate between tightly and loosely coupled architectures.
- Identify how Amazon Simple Queue Service (Amazon SQS) works and when to use it.
- Identify how Amazon Simple Notification Service (Amazon SNS) works and when to use it.
- Describe Amazon MQ.
- Decouple workloads by using Amazon SQS.

Module overview

Presentation sections

- Decoupling your architecture
- Decoupling applications with Amazon SQS
- Decoupling applications with Amazon SNS
- Decoupling a hybrid application with Amazon MQ

Knowledge checks

- 10-question knowledge check
- Sample exam question

Hands-on lab in this module

Guided lab

- Building Decoupled Applications by Using Amazon SQS

As a cloud architect designing application architectures:



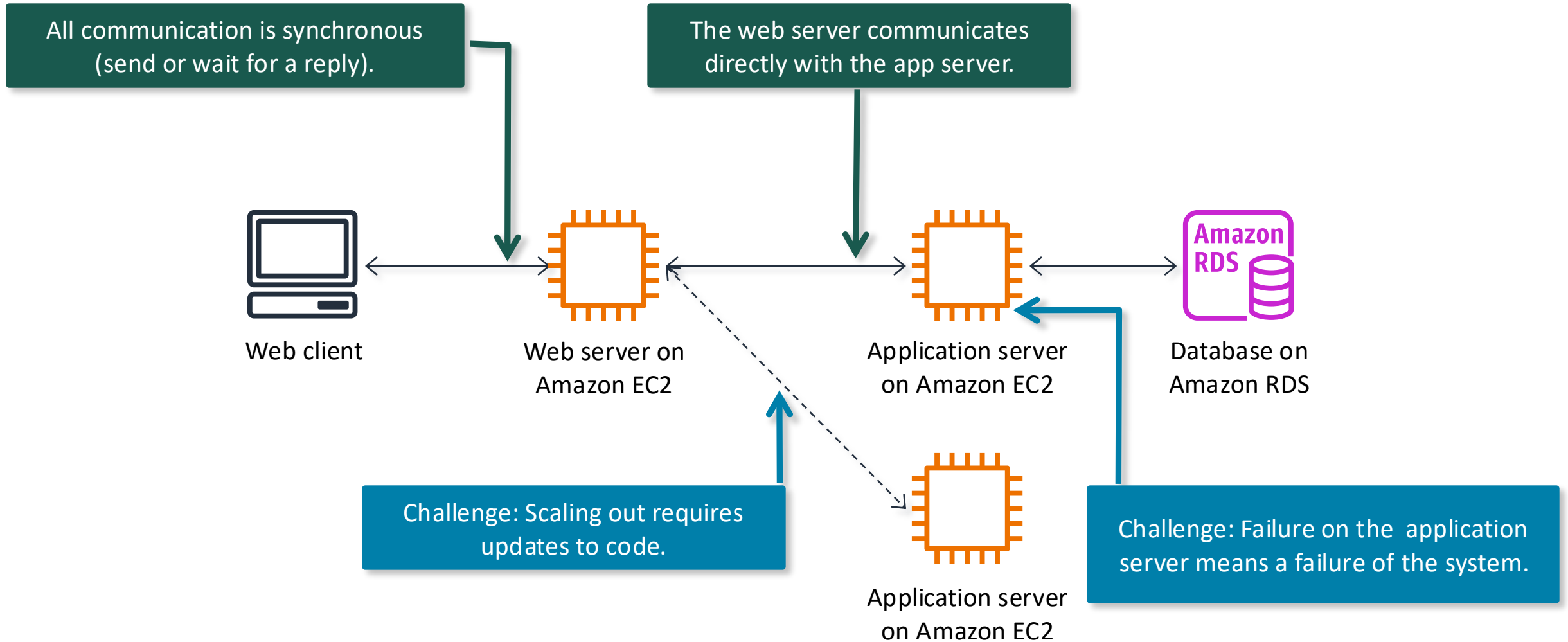
- I need to address potential performance bottlenecks so that the architecture scales as traffic increases.
- I need to limit the impact of failures across the application so that one component's failure does not bring down the application.
- I need to implement architectures in which changes can occur to one application component without affecting the other parts so that I can minimize downtime during maintenance.



Decoupling your architecture

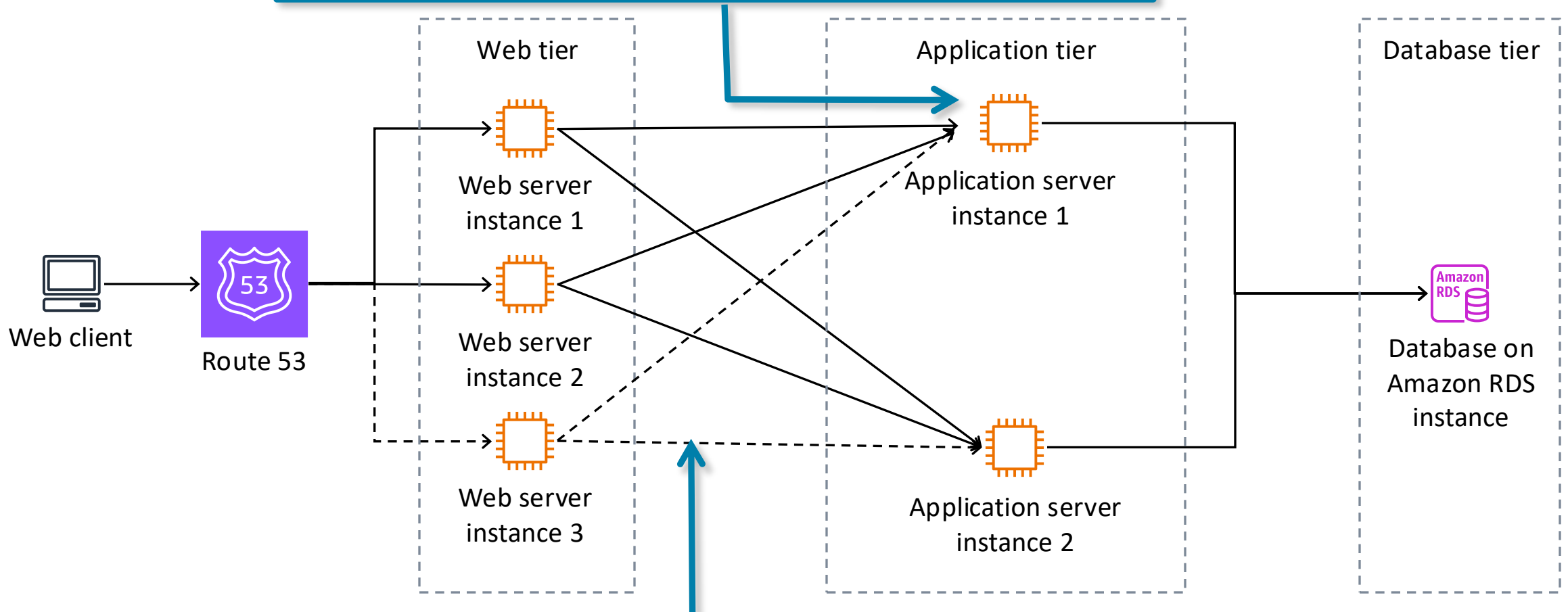
Building Decoupled Architectures

Tight coupling in a three-tier architecture



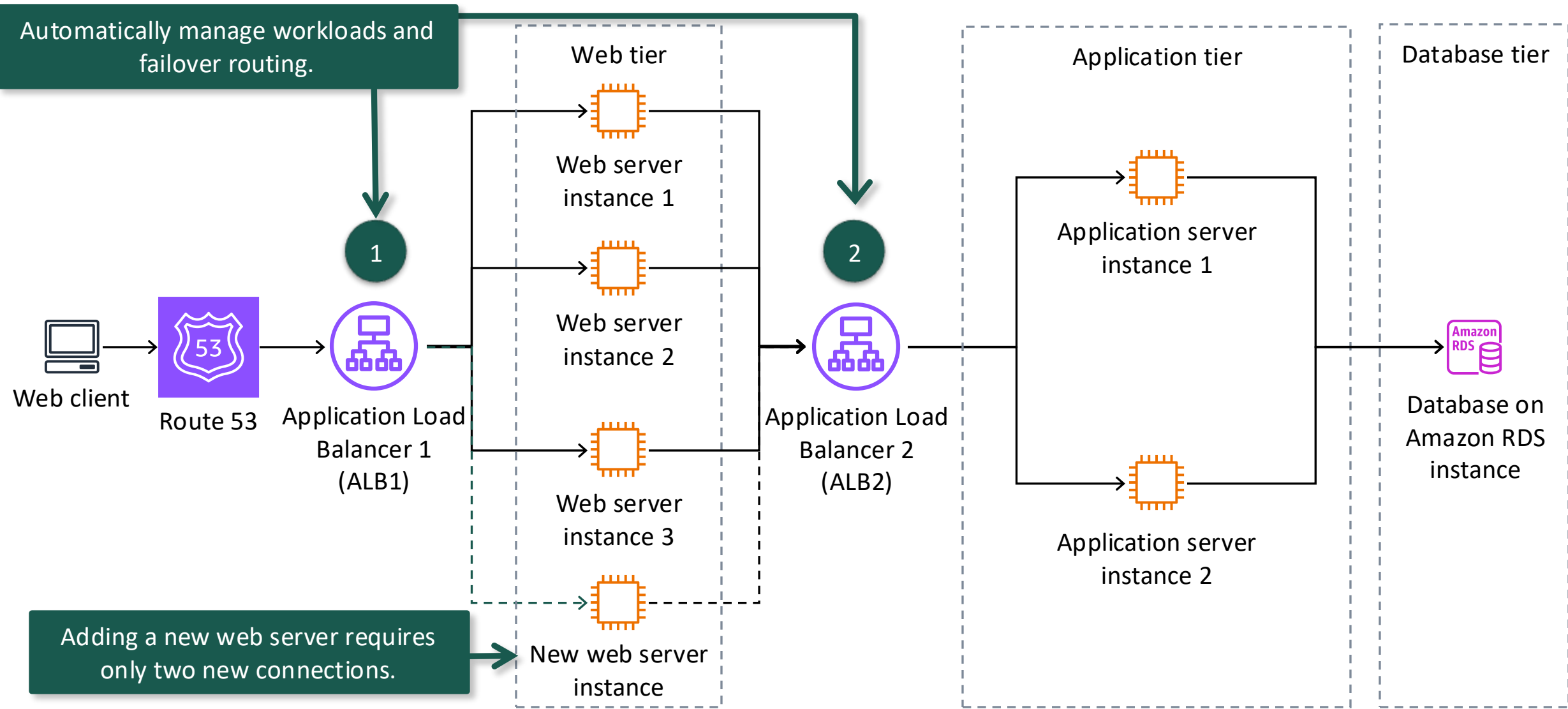
Tight coupling increases the complexity of scaling

An application outage impacts all connected web servers.

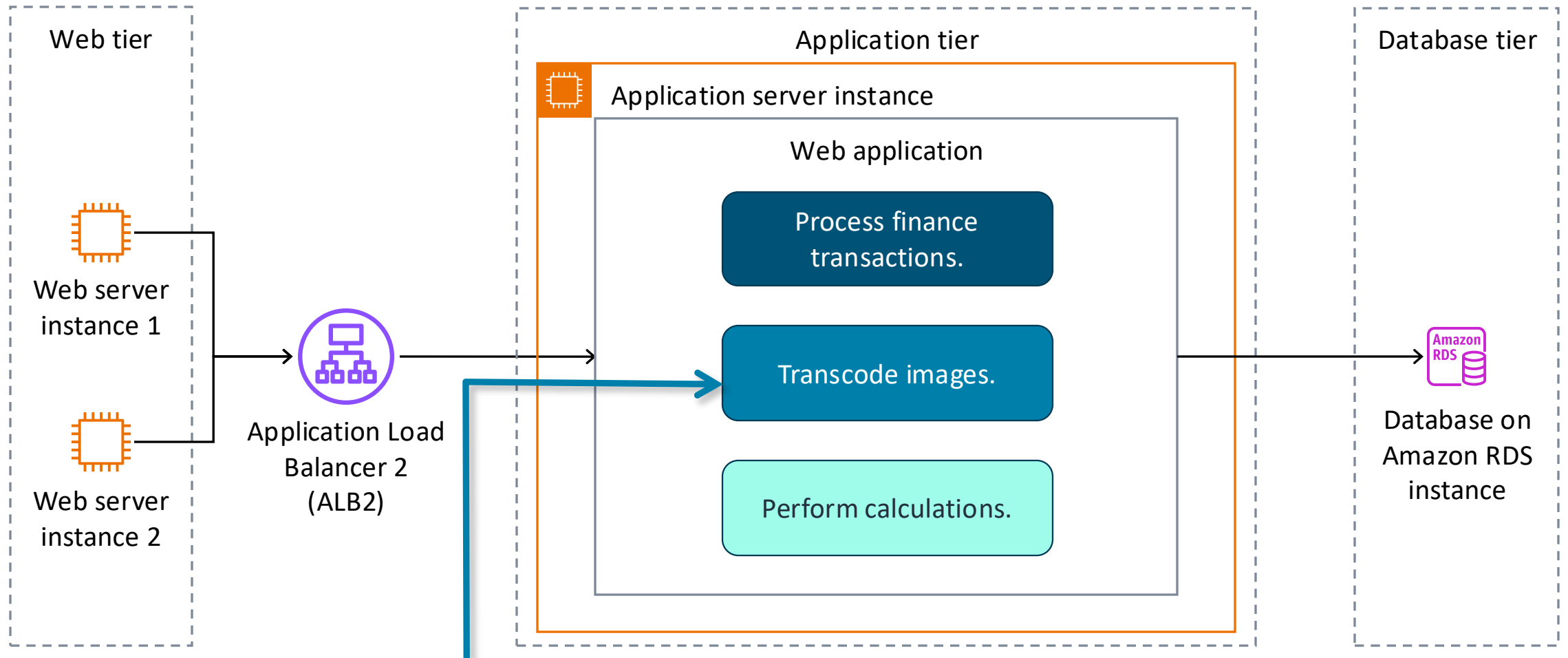


Each new server requires multiple connections that must be updated in code.

Loose coupling between tiers

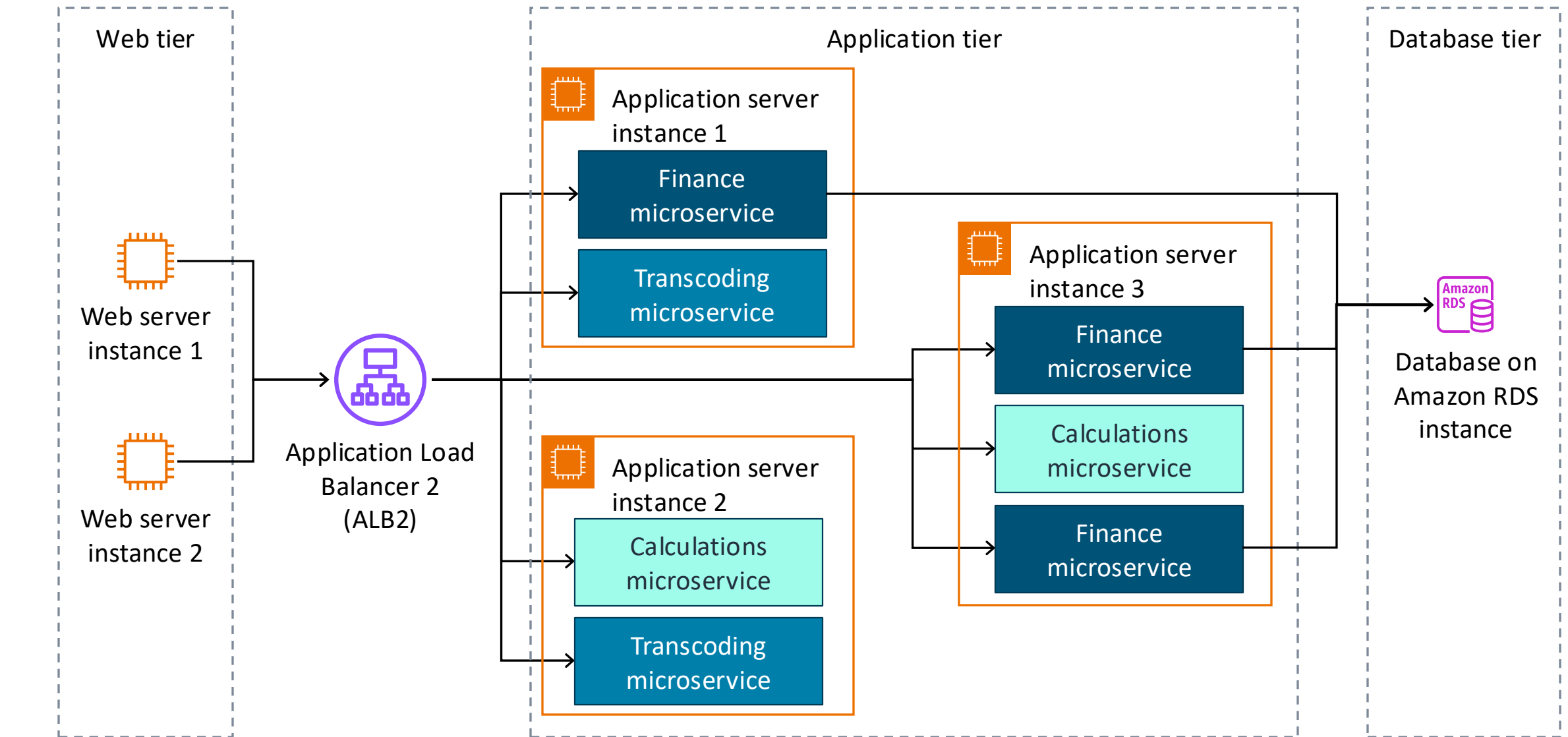


Tight coupling within an application

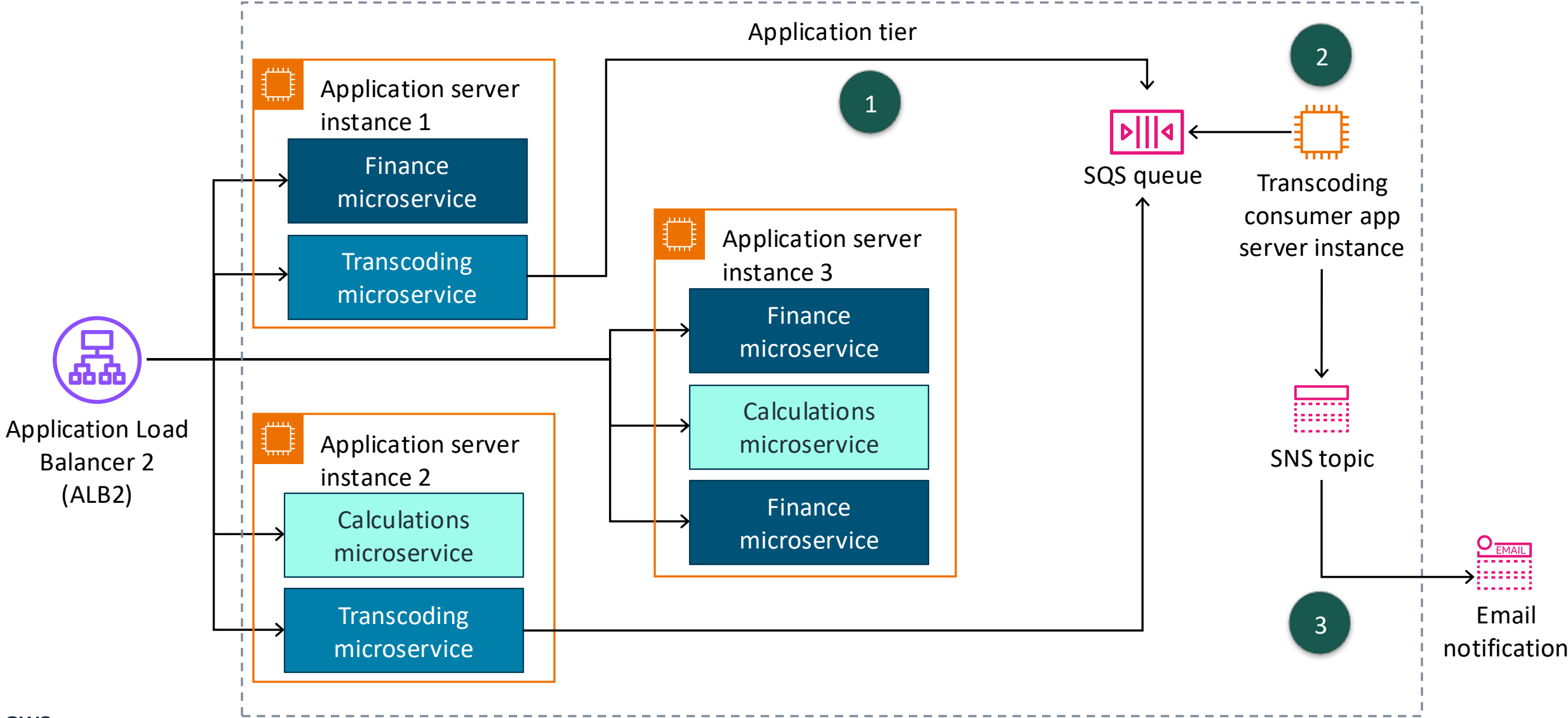


Challenge: The failure of one function can bring down the entire application.

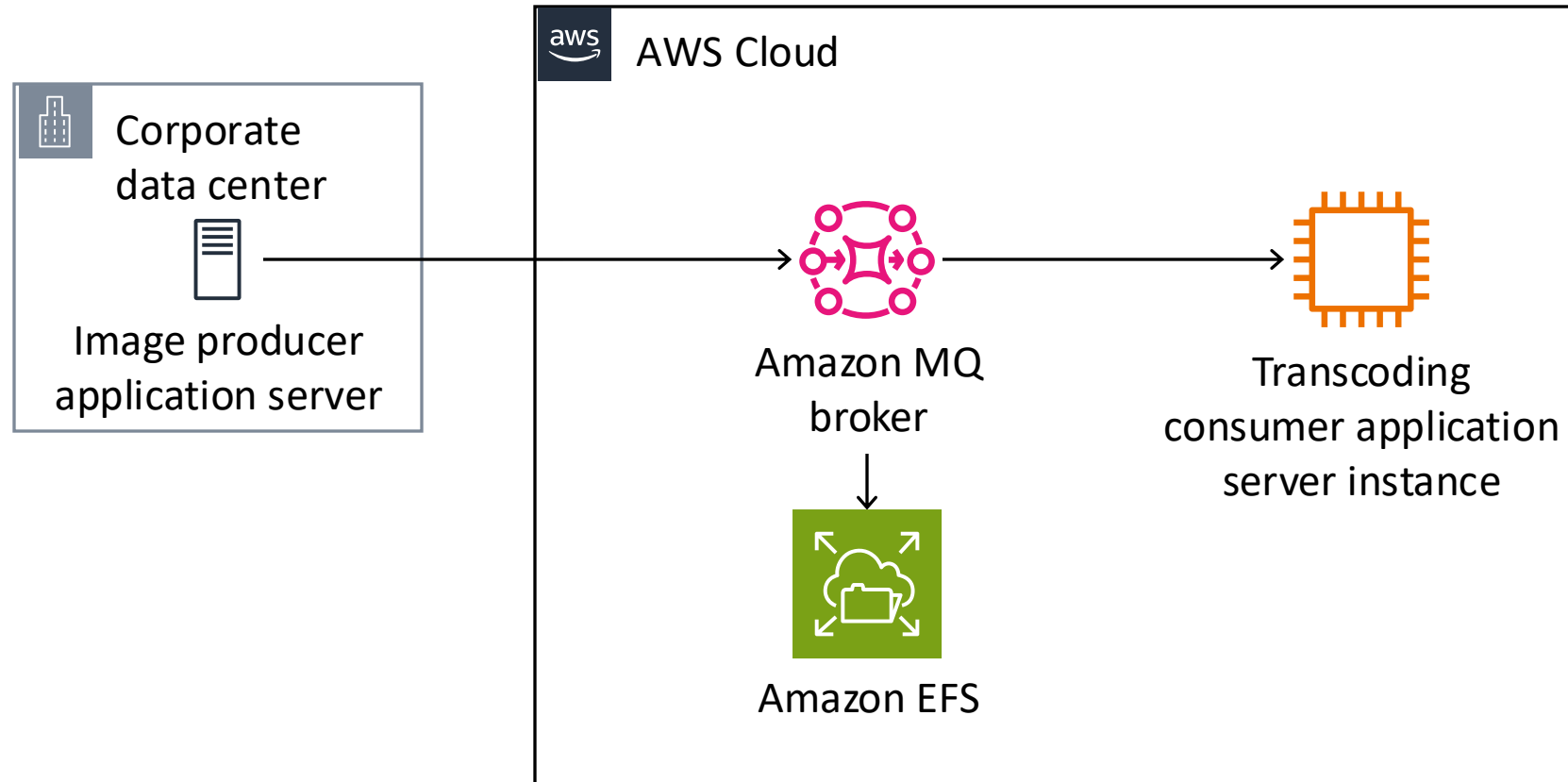
Loose coupling: Microservice architecture



Request offloading: Amazon SQS and Amazon SNS

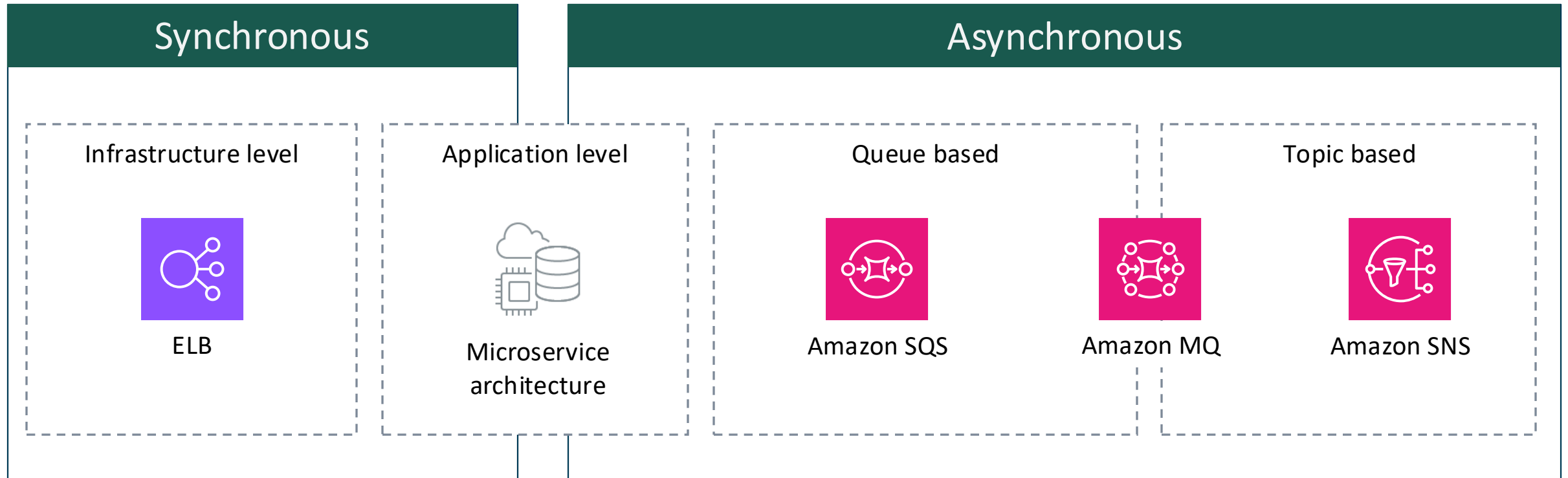


Loose coupling: Amazon MQ



Categories of solutions for loose coupling

Loose coupling solves tight integration problems and can be achieved through both synchronous and asynchronous solutions.



Key takeaways: Decoupling your architecture



- Tightly coupled systems are difficult to scale and introduce bottlenecks and single points of failure.
- A loosely coupled architecture removes direct dependencies between related components and permits scalability and resiliency.
- Loose coupling solutions divide infrastructure layers or application functions and typically introduce an intermediary component between them.
- Loose coupling solutions can be synchronous or asynchronous.
 - ELB is an example of a synchronous solution.
 - Amazon SQS, Amazon SNS, and Amazon MQ are examples of an asynchronous solution.

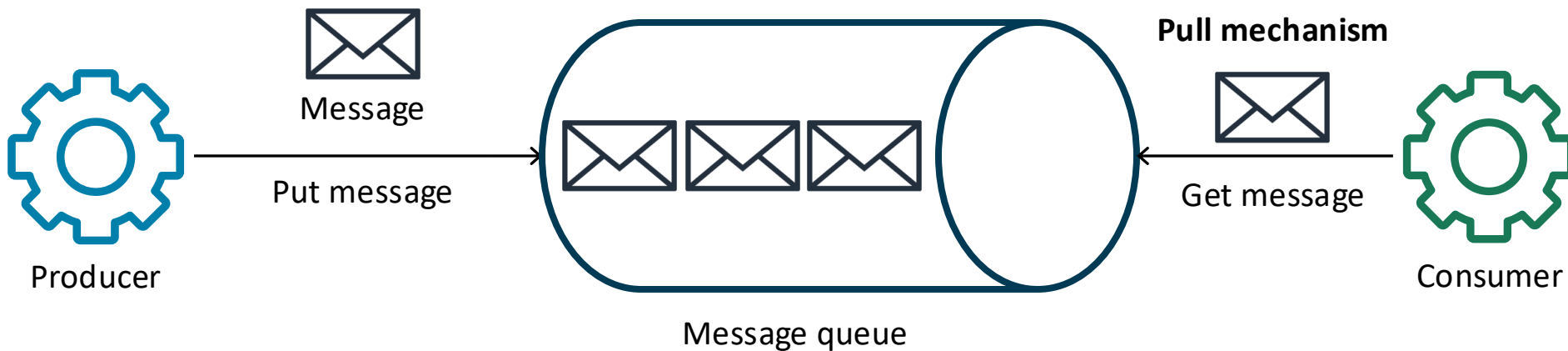


Decoupling applications with Amazon SQS

Building Decoupled Architectures

Point-to-point messaging

- You can decouple applications asynchronously by using point-to-point messaging.
- Using point-to-point messaging when the sending application sends a message to only one specific receiving application.
- The sending application is called a *producer*.
- The receiving application is called a *consumer*.
- Point-to-point messaging uses a *message queue* to decouple applications.



Amazon Simple Queue Service (Amazon SQS)



Amazon SQS

- Is a fully managed message queueing service
- Helps integrate and decouple distributed software systems and application components
- Provides highly available, secure, and durable message-queueing capabilities
- Provides an AWS Management Console interface and a web services API

Amazon SQS benefits

Fully managed

Avoid the need to manage messaging software or maintain infrastructure.

Reliability

Deliver large volumes of data without losing messages.

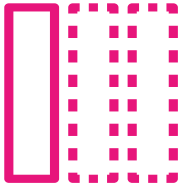
Security

Send sensitive data securely between applications.

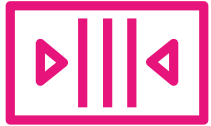
Scalability

Scale elastically based on usage.

Amazon SQS basic components



Message



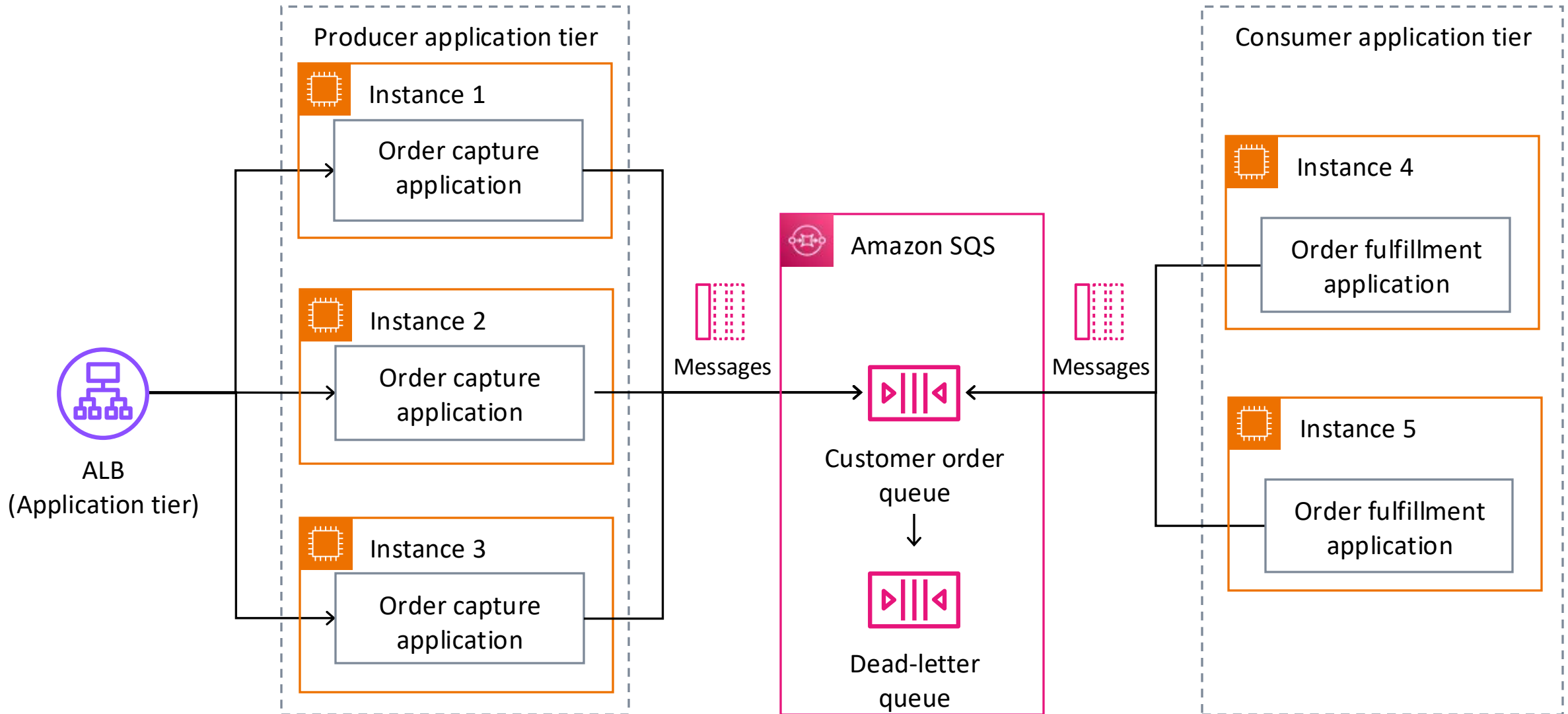
Queue



Dead-letter queue

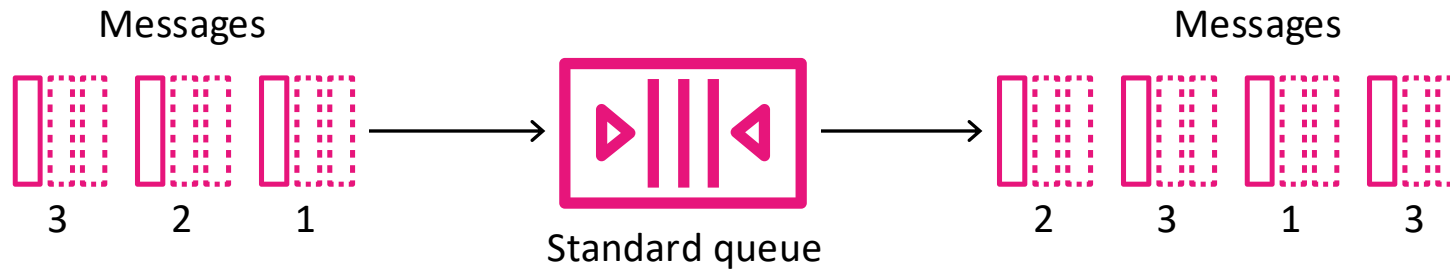
- A message can be up to 256 KB in size.
- A message remains in a queue until it is explicitly deleted or exceeds the queue's message retention period.
- Amazon SQS offers two types of queues: standard and first-in-first-out (FIFO)
- You can configure queue parameters, including the following:
 - Message retention period
 - Visibility timeout
 - Receive message wait time (short polling versus long polling)
- You can associate a dead-letter queue (DLQ) with any queue.
- A DLQ stores messages that cannot be consumed successfully.

Decoupling example: Amazon SQS



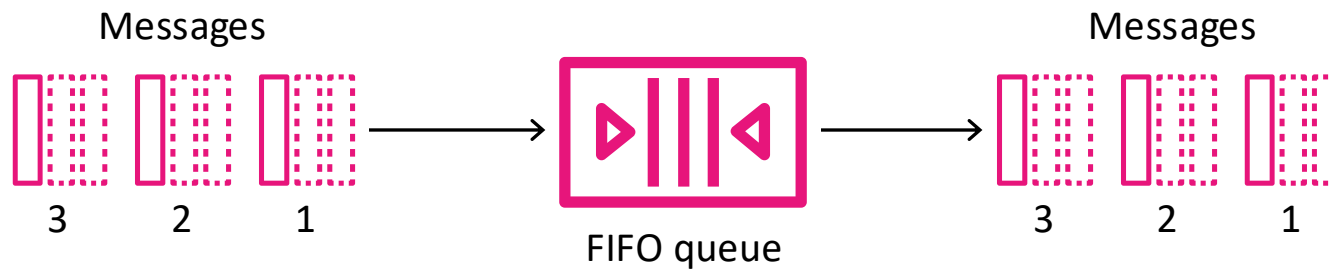
Queue types

Standard queue



- At-least-once delivery
- Best-effort ordering
- Nearly unlimited throughput

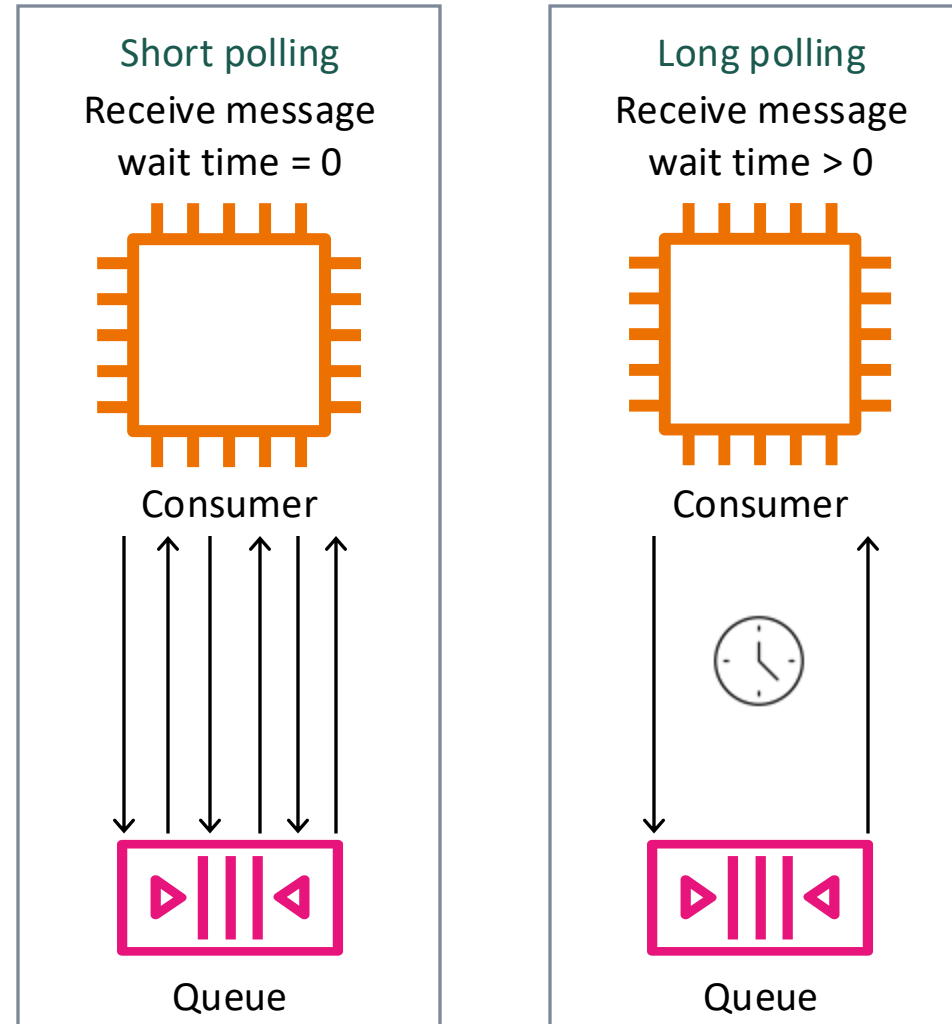
FIFO queue



- FIFO delivery
- Exactly once processing
- High throughput

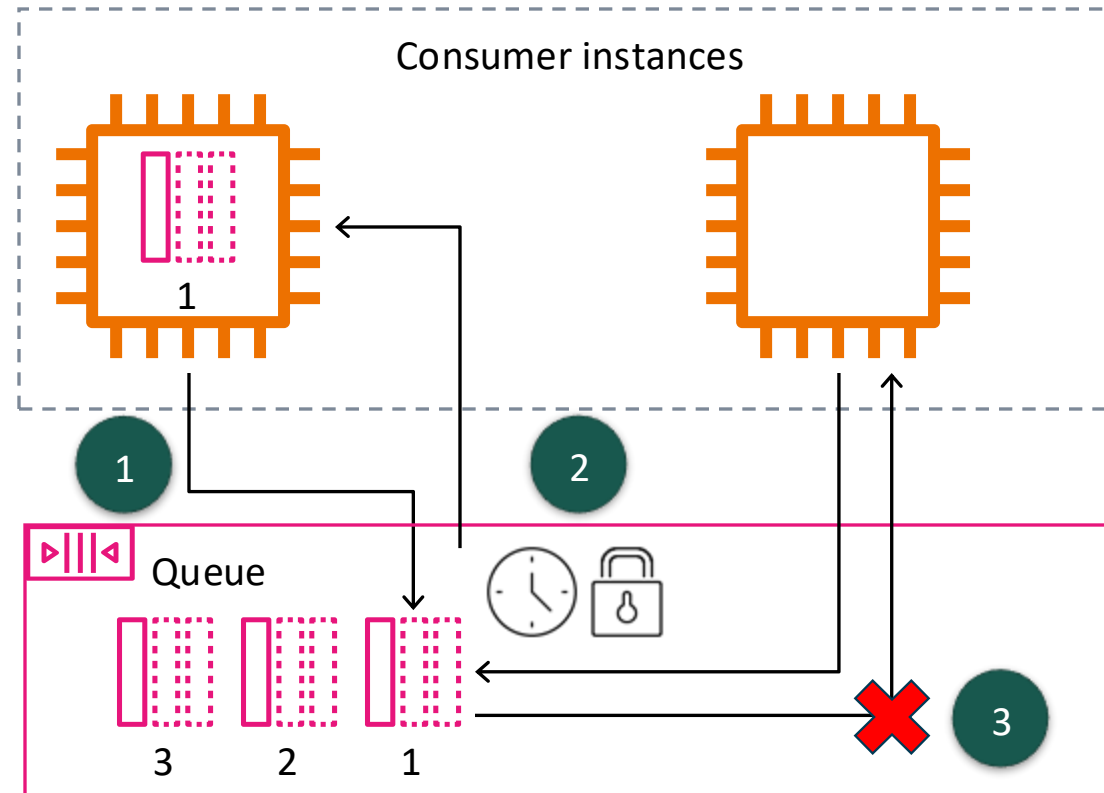
Queue configuration: Polling type

Choose the right polling type.

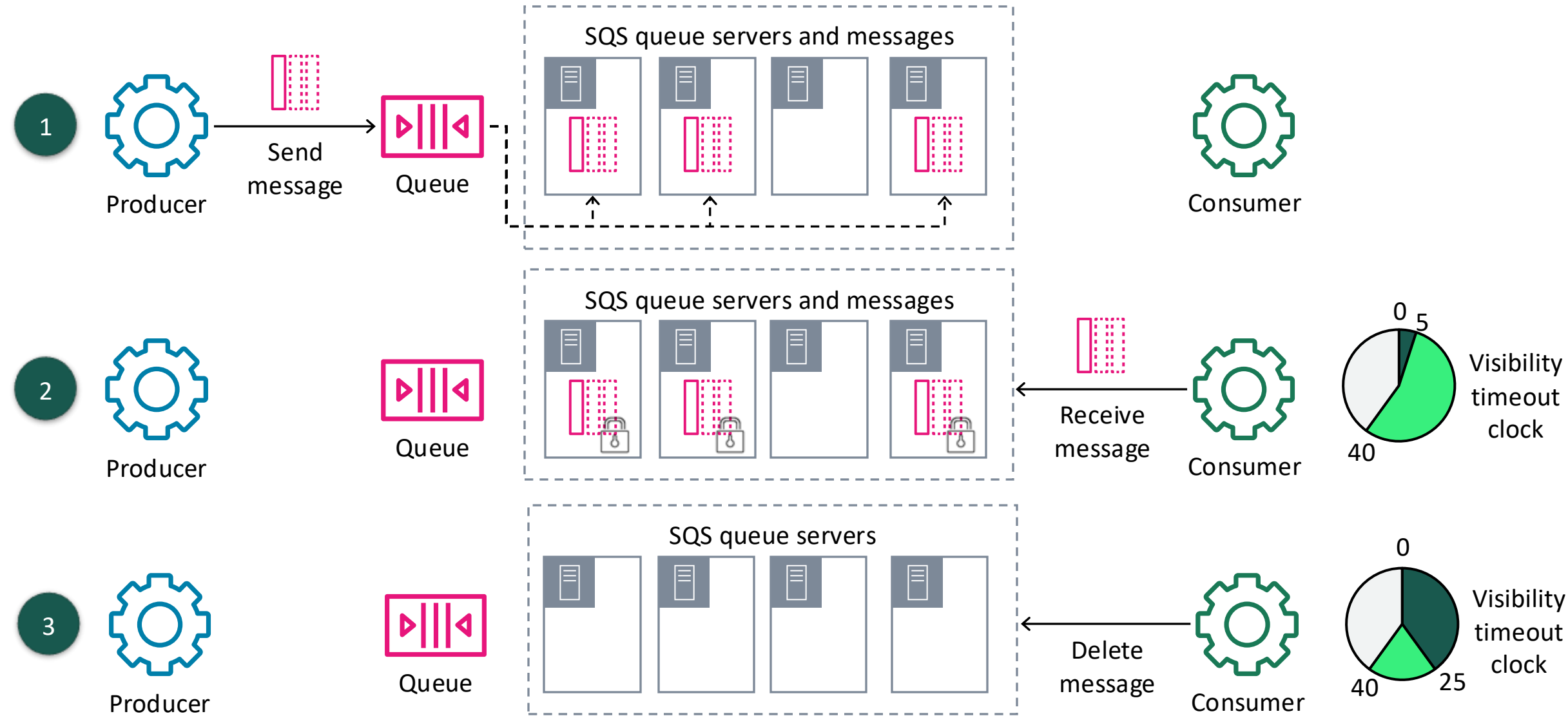


Queue configuration: Message visibility

Tune your visibility timeout.



How SQS message queueing works



Amazon SQS use cases

Work queues

Decouple components of a distributed application that process the same amount of work at a different rate.

Buffering and batch operations

Provide a buffer against traffic spikes or batch work for processing at a later time.

Request offloading

Queue requests to move slow operations off of interactive request paths.

Trigger Amazon EC2 Auto Scaling

Use SQS queues to help determine the load on an application and invoke scaling actions.

Key takeaways: Decoupling applications with Amazon SQS



- Amazon SQS is a fully managed message-queuing service that you can use to decouple application components so that they run independently.
- Amazon SQS supports standard and FIFO queues.
- Messages that cannot be processed can be sent to a dead-letter queue.
- Long polling helps reduce the cost of using Amazon SQS by reducing the number of empty responses to a consumer's receive message request.
- A producer sends a message to a queue. A consumer processes and deletes the message during the visibility timeout period.



Guided lab: Building Decoupled Applications by Using Amazon SQS (Amazon SQS lab)

Amazon SQS lab tasks



In this lab, you will perform the following main tasks:

- Create and configure an SQS queue.
- Use the queue in an application designed with decoupled functions.

Debrief: Amazon SQS lab

- How did you configure the SNS topic to send messages to the desired locations?
- How did you configure Amazon S3 to notify Amazon SNS when a new image is uploaded?

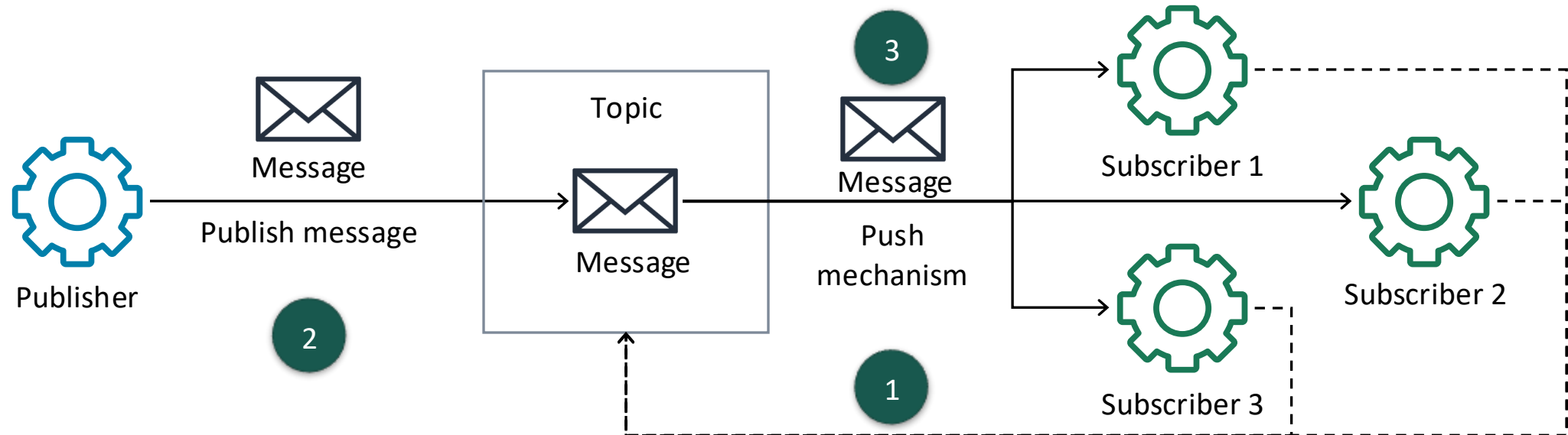


Decoupling applications with Amazon SNS

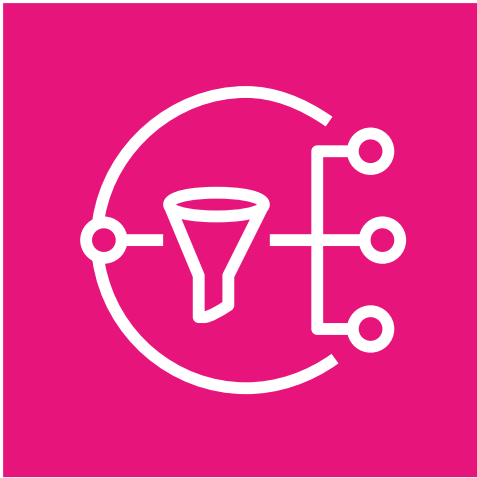
Building Decoupled Architectures

Publish/subscribe messaging

- You can decouple applications asynchronously by using publish/subscribe (pub/sub) messaging.
- Use pub/sub messaging when the sending application sends a message to multiple receiving applications and has little or no knowledge about the receiving applications.
- The sending application is called a *publisher*.
- The receiving application is called a *subscriber*.
- Pub/sub messaging uses a *topic* to decouple applications.



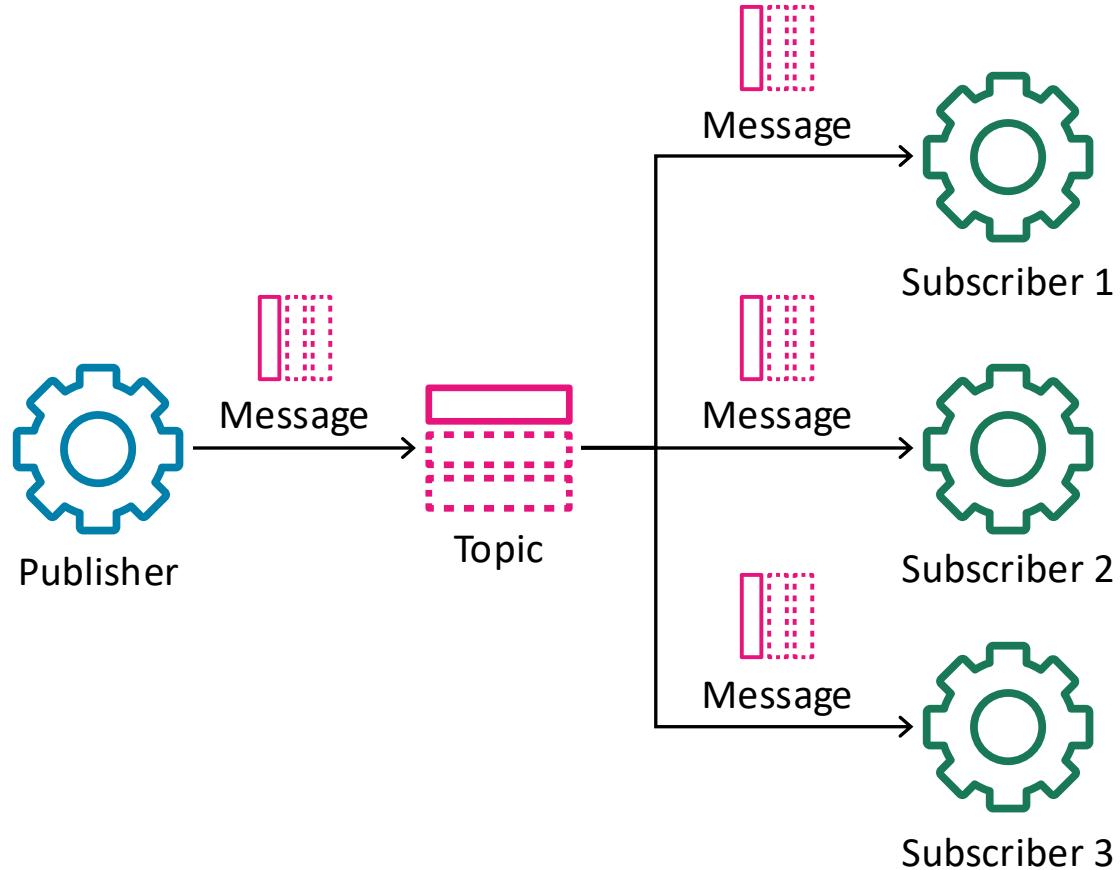
Amazon Simple Notification Service (Amazon SNS)



Amazon SNS

- Is a fully managed pub/sub messaging service
- Helps decouple applications through notifications
- Provides highly scalable, secure, and cost-effective notification capabilities
- Provides an AWS Management Console interface and a web services API

Types of subscribers

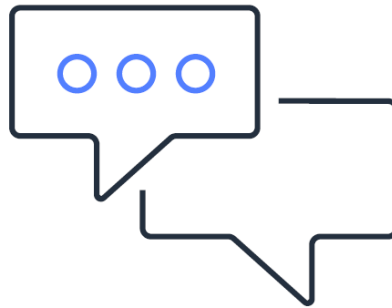


- Email destination
- Mobile text messaging destination
- Mobile push notifications endpoint
- HTTP or HTTPS endpoint
- AWS Lambda function
- SQS queue
- Amazon Kinesis Data Firehose delivery stream

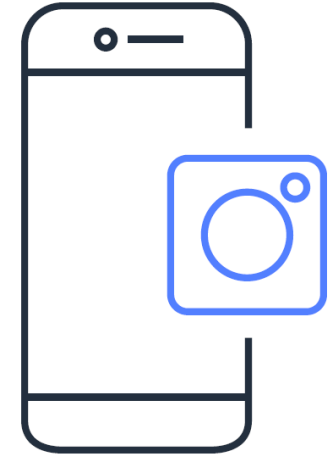
Amazon SNS use cases



Application and system alert
notification

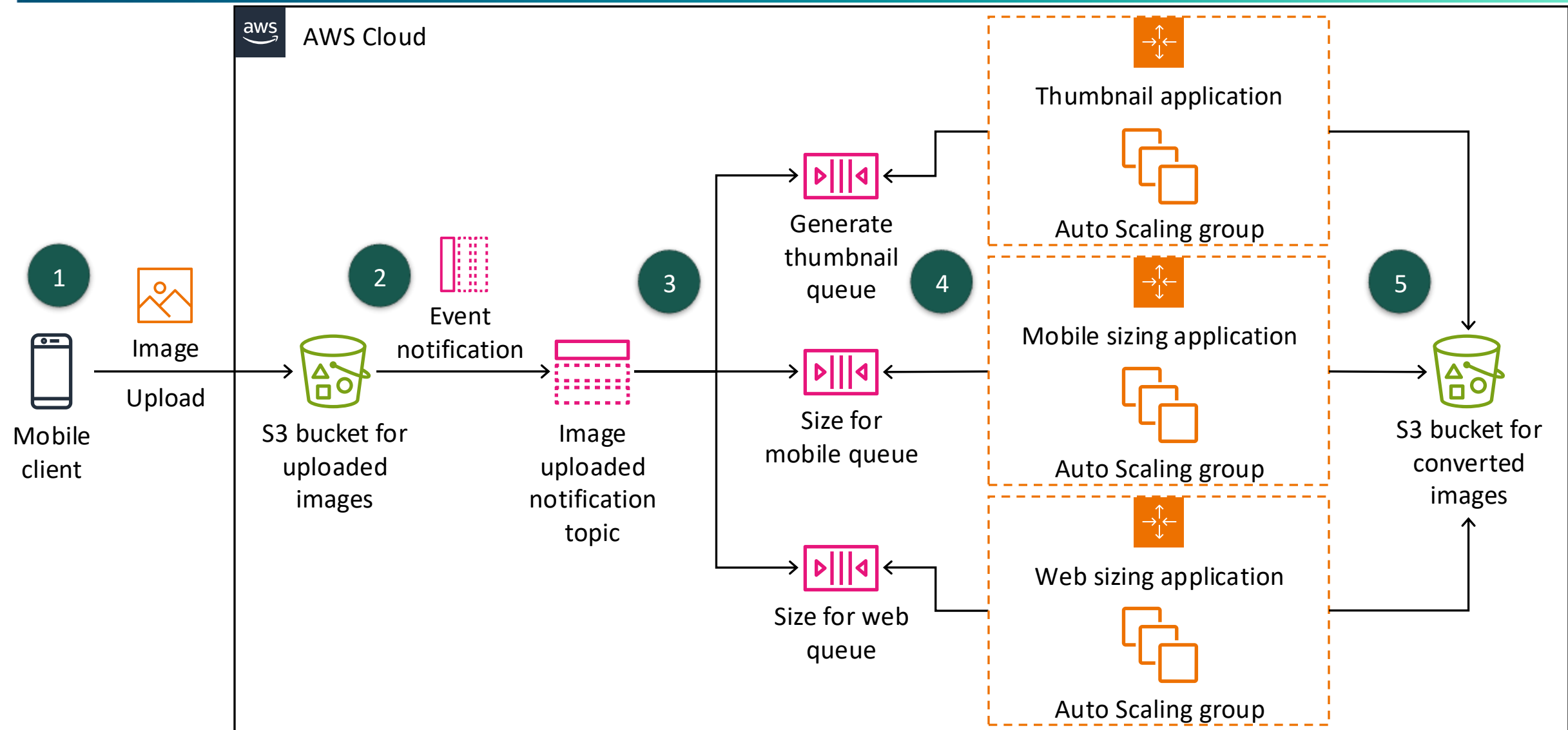


Email and text message
notification



Mobile push notification

Decoupling example: Using Amazon SQS with Amazon SNS



Amazon SNS considerations

Message publishing

- Single published message
- No recall options

Message delivery

- Use a *standard* topic if the message delivery order does not matter.
- Use a *FIFO* topic if an exact message delivery order is required.
- Customize the delivery policy of an HTTP or HTTPS endpoint to control the retry behavior.

Amazon SNS and Amazon SQS comparison

	Amazon SNS	Amazon SQS
Messaging Model	Publisher-Subscriber	Producer-Consumer
Distribution Model	One to many	One to one
Delivery Mechanism	Push (passive)	Pull (active)
Message Persistence	No	Yes

Key takeaways: Decoupling applications with Amazon SNS



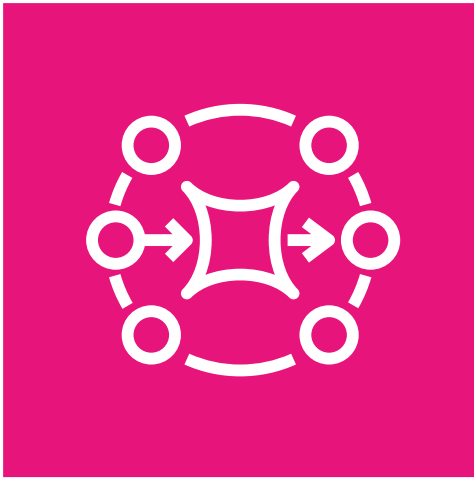
- Amazon SNS is a web service that you can use to set up, operate, and send notifications from the cloud.
- Amazon SNS follows the pub/sub messaging paradigm.
- To use Amazon SNS, you create a topic, create subscribers for the topic, and publish messages to the topic.
- You can use topics to decouple message publishers from subscribers and fanout messages to multiple recipients at one time.
- AWS services can publish messages to an SNS topic to invoke event-driven computing and workflows.



Decoupling a hybrid application with Amazon MQ

Building Decoupled Architectures

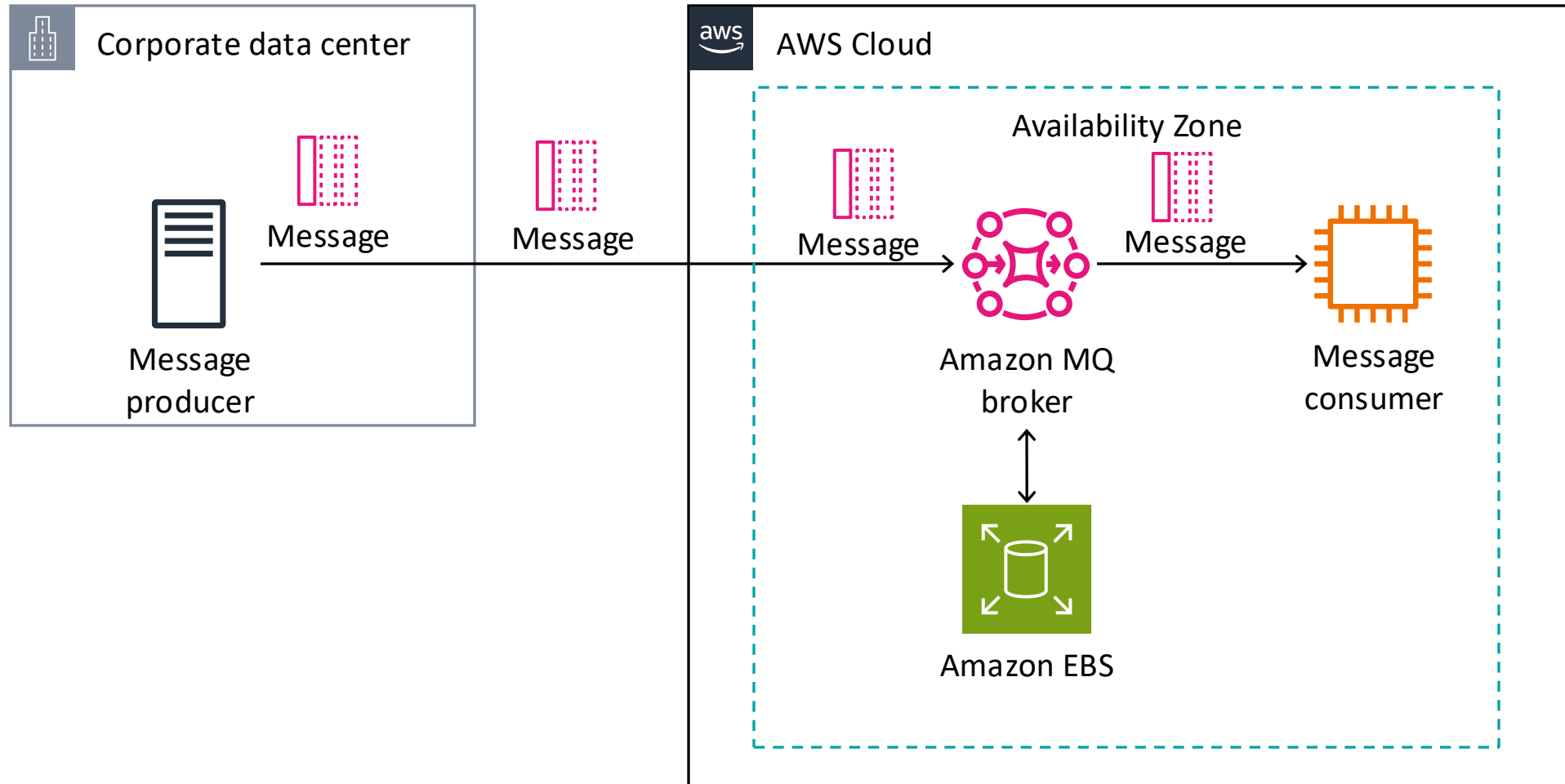
Amazon MQ



Amazon MQ

- Is a fully managed message broker service
- Facilitates the setup, operation, and management of an Apache ActiveMQ or RabbitMQ message broker in the AWS Cloud
- Provides a queue- and topic-based solution for loosely coupling applications
- Enables software applications and components to communicate by using various programming languages, operating systems, and formal messaging protocols

Amazon MQ use case: Hybrid cloud environment



Choosing the right solution for decoupling

	Amazon SQS	Amazon SNS	Amazon MQ
Applicability	Cloud-centered applications	Cloud-native applications	• Hybrid applications • Message broker migration
Messaging Model	Producer-Consumer	Publisher-Subscriber	• Producer-Consumer • Publisher-Subscriber
Programming API	Amazon SQS API	Amazon SNS API	• Industry standard message broker APIs
Pricing Model	Pay per request	Pay per request	• Pay per hour • Pay per GB

Key takeaways: Decoupling a hybrid application with Amazon MQ



- Amazon MQ is a managed service that you can use to set up and operate Apache ActiveMQ and RabbitMQ message brokers in the cloud.
- Amazon MQ is compatible with open standard messaging APIs and protocols.
- You can use Amazon MQ to integrate on-premises and cloud environments in a loosely coupled manner.
- You can also use Amazon MQ to migrate your on-premises open source message brokers to the AWS Cloud.



Module wrap-up

Building Decoupled Architectures

Module summary

This module prepared you to do the following:

- Differentiate between tightly and loosely coupled architectures.
- Identify how Amazon SQS works and when to use it.
- Identify how Amazon SNS works and when to use it.
- Describe Amazon MQ.
- Decouple workloads by using Amazon SQS.

Considerations for the café

- Discuss how you as a cloud architect might advise the café based on the key cloud architect concerns presented at the start of this module.



Module knowledge check



- The knowledge check is delivered online within your course.
- The knowledge check includes 10 questions based on material presented on the slides and in the slide notes.
- You can retake the knowledge check as many times as you like.

Sample exam question

A company must perform asynchronous processing and implement Amazon Simple Queue Service (Amazon SQS) as part of a decoupled architecture. The company wants to ensure that the number of empty responses from polling requests is kept to a minimum.

What should a solutions architect do to ensure that empty responses are reduced?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- SQS queue
- Empty responses kept to a minimum

Sample exam question: Response choices

A company must perform asynchronous processing and implement Amazon Simple Queue Service (Amazon SQS) as part of a decoupled architecture. The company wants to ensure that the number of empty responses from polling requests is kept to a minimum.

What should a solutions architect do to ensure that empty responses are reduced?

Choice	Response
A	Increase the maximum message retention period for the queue.
B	Increase the maximum receives for the redrive policy for the queue.
C	Increase the default visibility timeout for the queue.
D	Increase the receive message wait time for the queue.

Sample exam question: Answer

The answer is D.

Choice	Response
D	Increase the receive message wait time for the queue



Thank you

Corrections, feedback, or other questions?

Contact us at <https://support.aws.amazon.com/#/contacts/aws-academy>.